

**YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND
MANAGEMENT**

**A CONSTITUENT UNIT OF YENEPOYA (DEEMED TO BE UNIVERSITY)
BALMATTA, MANGALORE**

HIGH LEVEL DESIGN (HLD)

Multi-Cloud Hybrid Strategy for Agriculture

SUBMITTED BY:

Muhammed Umer Sufaidh

III Year BCA

(Artificial Intelligence, Cloud Computing and DevOps)With IBM&TCS

REGISTER NUMBER: 23BCAICD085

UNDER THE GUIDANCE OF

Ms.Sindhu

[Faculty Guide]

INDUSTRY IBM MENTOR: Sumit Kumar Shukla

DEPARTMENT OF COMPUTER SCIENCE

**IN PARTIAL FULFILMENT OF THE REQUIREMENT FOR THE AWARD
OF THE DEGREE OF**

BACHELOR OF COMPUTER APPLICATIONS

MAY 2026

HIGH LEVEL DESIGN (HLD)

Table of Contents

- 1. Introduction 4
 - 1.1 Scope of the Document 4
 - 1.2 Intended Audience 4
 - 1.3 System Overview 4
- 2. System Design 5
 - 2.1 Application Design 5
 - 2.2 Process Flow 5
 - 2.3 Information Flow 6
 - 2.4 Components Design 6
 - 2.5 Key Design Considerations 7
 - 2.6 API Catalogue 7
- 3. Data Design 8
 - 3.1 Data Model 8
 - 3.2 Data Access Mechanism 8
 - 3.3 Data Retention Policies 9
 - 3.4 Data Migration 9
- 4. Interfaces 9
- 5. State and Session Management 9
- 6. Caching 10
- 7. Non-Functional Requirements 10
 - 7.1 Security Aspects 10
 - 7.2 Performance Aspects 10
- 8. References 11

1. Introduction

1.1 Scope of the Document

This High Level Design (HLD) document describes the architecture of the Multi-Cloud Hybrid Backup Strategy for Agriculture. The system backs up agriculture-related data files (crop reports, soil analysis, weather logs, irrigation schedules, pesticide records) to multiple storage destinations using a Python-based failover mechanism. This document covers the overall system architecture, component interactions, data flows, and non-functional requirements.

1.2 Intended Audience

- BCA/BSc students and developers working on cloud computing projects
- University faculty and industry mentors reviewing the design
- Cloud engineers and DevOps professionals evaluating the architecture
- Testers responsible for validating backup reliability and failover logic

1.3 System Overview

The Agriculture Multi-Cloud Backup System (PRJN26-055) is a Python-based application that simulates a professional multi-source backup strategy for agricultural data. The system maintains a source directory of agriculture documents and attempts to back them up to three destinations in failover order:

- Local Storage: A timestamped folder on the local machine
- Google Drive: Via the Google Drive Desktop sync folder (auto-synced to cloud)
- AWS S3: Via the boto3 SDK to an S3 bucket in the ap-south-1 region

If local backup fails, the system automatically escalates to Google Drive. If that also fails, it escalates to AWS S3. All backup results are logged to `backup_log.txt` and a summary is saved to `last_backup_summary.json`. A Streamlit web dashboard (`app.py`) provides a user-friendly interface to trigger backups and view results.

2. System Design

2.1 Application Design

The system is composed of two primary application layers:

Component	File / Module	Role
Core Backup Engine	backup.py	All backup logic, failover controller, sample data generator
Web Dashboard	app.py (Streamlit)	User interface to trigger backups and view logs/summaries
Configuration	CONFIG dict (backup.py)	Source path, destination paths, credentials, log file
Logging	backup_log.txt	Timestamped event log for all backup operations
Backup Summary	last_backup_summary.json	JSON record of last run: success/failure per destination

2.2 Process Flow

The following sequence describes the end-to-end backup process triggered by the user:

- Step 1 – User initiates backup via Streamlit dashboard (app.py) or CLI (backup.py)
- Step 2 – System reads source directory (./agriculture_data/) containing agriculture files
- Step 3 – Backup attempt to Local Storage: copies entire directory to ./local_backup/backup_{timestamp}/
- Step 4 – If Local succeeds, cloud backups also run for redundancy. If Local fails, the system logs the failure and escalates.
- Step 5 – Backup attempt to Google Drive: copies files to the local Google Drive Desktop sync folder (G:\My Drive\AgriBackup\), which auto-syncs to the cloud
- Step 6 – If Google Drive fails, the system escalates to AWS S3
- Step 7 – Backup attempt to AWS S3: uploads files using boto3 to s3://agri-backup-umer-2026/agriculture_backup/{timestamp}/
- Step 8 – Final summary is generated: success/failure per destination is printed, logged, and saved to last_backup_summary.json
- Step 9 – If ALL three methods fail, a CRITICAL alert is raised in the log

2.3 Information Flow

The information flow between system layers is as follows:

Source	Direction	Destination	Data / Signal
User (Dashboard/CLI)	→	app.py / backup.py	Trigger command (button click or menu choice)
backup.py	→	agriculture_data/	Read source files

Source	Direction	Destination	Data / Signal
backup.py	→	local_backup/	Copied file tree (shutil.copytree)
backup.py	→	G:\My Drive\ AgriBackup	Copied files (shutil.copy2)
backup.py	→	AWS S3 Bucket	Uploaded files (boto3 s3.upload_file)
backup.py	→	backup_log.txt	INFO/WARNING/ERROR/ CRITICAL log entries
backup.py	→	last_backup_summary.json	JSON: {local, google_drive, aws_s3, timestamp}

2.4 Components Design

backup_local()

Uses `shutil.copytree()` to replicate the entire source directory to a new timestamped subfolder under `./local_backup/`. Returns `True` on success, `False` on failure. Raises `FileNotFoundError` if source directory is missing.

backup_google_drive()

Copies each file individually to the Google Drive Desktop sync folder using `shutil.copy2()`. The Google Drive Desktop application handles actual cloud synchronization. Returns `True` on success, `False` on failure.

backup_aws_s3()

Uses `boto3` to create an S3 client with explicit credentials and uploads each file to the configured bucket under a timestamped key prefix (`agriculture_backup/{timestamp}/{filename}`). Handles `NoCredentialsError` and `ClientError` per file and logs partial failures.

run_backup_with_failover()

The main orchestrator. Calls all three backup functions in sequence. Logs start/end banners. Tracks results in a results dict. If all three fail, logs a CRITICAL alert. Always writes `last_backup_summary.json` at the end regardless of outcome.

create_sample_agriculture_data()

Generates five representative agriculture files for testing: `crop_report_april2026.txt`, `soil_analysis.txt`, `weather_log.txt`, `irrigation_schedule.txt`, `pesticide_usage.txt`.

2.5 Key Design Considerations

- Redundancy: All three backup destinations run every time (not just on failure), ensuring maximum data copies
- Failover: If Local fails, the system continues to cloud without stopping
- Atomicity: Each backup function is fully self-contained and returns a boolean result

- **Observability:** Python logging module writes timestamped events to both file and console
- **Portability:** No cloud-specific SDK is mandatory for local backup; boto3 is imported only inside the S3 function
- **Testability:** Sample data generator allows full end-to-end testing without real agriculture files
- **Configurability:** All paths, credentials, and bucket names are centralized in the CONFIG dictionary

2.6 API Catalogue

The Streamlit dashboard (app.py) exposes the following user-facing actions. These are not REST APIs but internal function calls wrapped by Streamlit button events.

Button / Action	Function Called	Description
Create Sample Data + Run Full Backup	create_sample_agriculture_data() + run_backup_with_failover()	Generates test data and runs all three backup destinations
Run Backup Only	run_backup_with_failover()	Backs up existing source data
Test Local Backup Only	backup_local()	Tests only the local storage backup path
Show Backup Log	Reads backup_log.txt	Displays full log file in a text area widget
Show Last Backup Summary	Reads last_backup_summary.json	Displays JSON summary of the most recent backup run

3. Data Design

3.1 Data Model

The system operates on flat-file agriculture documents. There is no relational database. The data entities are:

File / Entity	Type	Location	Description
crop_report_april2026.txt	Plain text	./agriculture_data/	Crop name, yield, season, status
soil_analysis.txt	Plain text	./agriculture_data/	pH, nitrogen, phosphorus, moisture
weather_log.txt	Plain text	./agriculture_data/	Date, temperature, rainfall, humidity
irrigation_schedule.txt	Plain text	./agriculture_data/	Field-wise watering schedule
pesticide_usage.txt	Plain text	./agriculture_data/	Chemical name, quantity, dates applied
last_backup_summary.json	JSON	./ (project root)	Backup result flags and ISO timestamp
backup_log.txt	Plain text (log)	./ (project root)	Timestamped operational log

3.2 Data Access Mechanism

- Source files are read using `os.walk()` to traverse all subdirectories
- Local backup uses `shutil.copytree()` for atomic directory-level copy
- Google Drive files are copied using `shutil.copy2()` (preserves metadata)
- AWS S3 files are uploaded using `boto3 s3.upload_file()` per file
- Log file is opened in append mode by Python's `logging.FileHandler`
- Summary JSON is written using `json.dump()` after every backup run

3.3 Data Retention Policies

- Local backups accumulate in `./local_backup/` indefinitely (no auto-deletion implemented)
- Each local backup is stored under a unique timestamp folder (`backup_YYYYMMDD_HHMMSS`)
- AWS S3 objects are stored under timestamped key prefixes; no lifecycle policy is configured at this stage
- Google Drive files overwrite previous copies with the same filename (no versioning)
- `last_backup_summary.json` is overwritten after every run (single record retained)
- `backup_log.txt` is appended on every run (cumulative history retained)

3.4 Data Migration

No data migration is currently required. The system is designed for a single-node Python deployment. Future migration considerations include:

- Converting source files to structured JSON or CSV for cloud-native storage

- Migrating to a database (e.g., PostgreSQL) for tracking backup metadata
- Integrating with cloud-native services (AWS RDS, Google BigQuery) for analytics
- Adding Google Drive API (OAuth2 + service account) for programmatic upload instead of Desktop sync

4. Interfaces

4.1 User Interfaces

- Streamlit Web Dashboard (app.py): Browser-based UI with five action buttons, configuration panel, log viewer, and summary viewer. Run using: streamlit run app.py
- Command-Line Interface (backup.py): Interactive menu with three options: (1) Create sample data + full backup, (2) Full backup only, (3) Local backup test only

4.2 Storage Interfaces

- Local File System: OS-level file I/O via Python os, shutil, and pathlib modules
- Google Drive: Local sync folder interface via Google Drive Desktop application (G:\My Drive\AgriBackup)
- AWS S3: boto3 SDK interface using IAM credentials (access key + secret key). Region: ap-south-1. Bucket: agri-backup-umer-2026

4.3 Logging Interface

- Python logging module with dual handlers: FileHandler (backup_log.txt) and StreamHandler (terminal)
- Log format: %(asctime)s [%(levelname)s] %(message)s
- Log levels used: INFO, WARNING, ERROR, CRITICAL

5. State and Session Management

The system is stateless between runs. No session data is maintained in memory across backup executions. State is persisted externally:

- last_backup_summary.json stores the result of the most recent backup (local/gdrive/s3 success flags + ISO timestamp)
- backup_log.txt stores cumulative historical state of all backup events
- Streamlit session state is not explicitly used; all data is read directly from files on each button click
- Each backup run is independent; there is no dependency on the result of previous runs

6. Caching

No dedicated caching mechanism is implemented in this version. The following implicit caching behaviors exist:

- Python module-level CONFIG dict is loaded once at startup and reused across all function calls
- boto3 S3 client is created fresh on each backup_aws_s3() call (no connection pooling)
- Streamlit may cache static assets in the browser (CSS, JS)

Future improvement: boto3 client could be created once and passed as a parameter to avoid repeated authentication overhead.

7. Non-Functional Requirements

7.1 Security Aspects

- AWS credentials (access key and secret key) are currently stored in plaintext in CONFIG dict – not recommended for production; should use AWS IAM roles or environment variables
- Google Drive credentials path points to a local service account JSON file – file must be kept secure and not committed to version control
- Google Drive Folder ID is a non-sensitive string identifying the target folder
- No user authentication is implemented in the Streamlit dashboard in this prototype
- All file operations are limited to the local filesystem within the defined source and backup directories
- Recommendation: Use python-dotenv or AWS Secrets Manager for credential management in production

7.2 Performance Aspects

- Local backup performance depends on file count and size; `shutil.copytree()` is synchronous and blocking
- Google Drive sync latency depends on the Google Drive Desktop app and network speed
- AWS S3 upload speed depends on internet bandwidth and file size; uploads are per-file (not multipart) in this implementation
- boto3 upload is sequential (one file at a time); parallel uploads via `ThreadPoolExecutor` would improve performance for large datasets
- Streamlit UI is blocking during backup execution; a spinner is shown to indicate progress
- For the current dataset size (5 small text files), all operations complete in under 5 seconds on average
- Scalability recommendation: For large agriculture datasets, implement chunked uploads and asyncio-based concurrency

8. References

- Python Official Documentation – shutil module: <https://docs.python.org/3/library/shutil.html>
- Python Official Documentation – logging module: <https://docs.python.org/3/library/logging.html>
- AWS boto3 Documentation – S3 Client: <https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/s3.html>
- Google Drive Desktop Application: <https://support.google.com/drive/answer/7329379>
- Streamlit Documentation: <https://docs.streamlit.io>
- AWS S3 Free Tier: <https://aws.amazon.com/s3/pricing/>
- Google Drive API: <https://developers.google.com/drive/api/guides/about-sdk>
- Project Repository: Google Drive – <https://drive.google.com/drive/folders/1D4zACdJ3684-sJjnyuYLz9bWaNcAM6KO>
- Project Code: PRJN26-055 | Domain: AI, Cloud Computing and DevOps